
SQL OPTIMIZATION

1

INTERVIEW QUESTION



Interview Question

Q: Which performs better — IN or EXISTS, and why?

Asked by: Oracle, Capgemini Elite Hiring

Answer:

- **IN:** Loads the **entire subquery result** and compares each row from the outer query. Performance suffers with **large result sets** or when **NULLs** are involved.
- **EXISTS:** Returns **TRUE immediately** upon finding the **first match** — great for **correlated subqueries** or large datasets.
- **Use IN** when the list is **small, static, and non-null**.
- **Use EXISTS** for **dynamic, relational, or row-wise checks**.



Why It Matters:

Despite similar logic, IN and EXISTS are fundamentally different in execution:

Aspect	IN	EXISTS
Memory Usage	Higher (materializes full result set)	Lower (evaluates row-by-row)
Early Exit	✗ No	✓ Yes
NULL Sensitivity	✗ Can fail silently	✓ Safe
Correlated Subqueries	✗ Inefficient	✓ Optimized
Use Indexes	Sometimes	Often (with proper filtering)

In modern SQL engines like **PostgreSQL**, **SQL Server**, **Oracle**, and **Snowflake**, query planners **may auto-optimize IN into a semi join** — but not always.

In **distributed engines like Spark**, EXISTS-style behavior should be written using **LEFT SEMI JOIN** to gain performance benefits.



Examples:



Slower Query (Using IN)

```
SELECT name FROM employees  
WHERE department_id IN (SELECT id FROM  
departments);
```

◆ If departments has 100K+ rows or NULLs → poor performance, or empty result set



Optimized Query (Using EXISTS)

```
SELECT name FROM employees e  
WHERE EXISTS (  
  SELECT 1 FROM departments d  
  WHERE d.id = e.department_id  
);
```



Leverages indexes



Short-circuits after first match



Safe from NULL-related issues



Optimization Tips:

◆ Use EXISTS for:

- Large or correlated datasets
- Presence/absence checks
- Avoiding NULL pitfalls
- Filtering with index-based lookups

◆ Use IN for:

- Small static lists like IN ('A', 'B', 'C')
- Columns with enforced NOT NULL constraints
- Dimension table lookups that fit in memory

◆ **Avoid pitfalls:**

- IN returns UNKNOWN if NULLs exist in subquery
- EXISTS can behave unexpectedly if not properly correlated
- IN may cause **full scan + sort + deduplication** depending on engine
- **Spark tip:** Use LEFT SEMI JOIN instead of EXISTS for optimized plan

◆ **Check Execution Plans:**

- OLAP engines: Look for **broadcasts, shuffles, and sorts**
- OLTP engines: Look for **index usage, nested loops, and hash joins**



Real-World Scenario:

You're working with two enterprise datasets:

- login_attempts → 50M rows
- user_profiles → 5M rows

❌ Inefficient Query with IN:

```
SELECT user_id FROM login_attempts  
WHERE user_id IN (SELECT id FROM user_profiles);
```

➖ Full materialization of user_profiles → High memory usage and latency

✅ Efficient Query with EXISTS:

```
SELECT user_id FROM login_attempts la  
WHERE EXISTS (  
  SELECT 1 FROM user_profiles up  
  WHERE up.id = la.user_id  
);
```

- ✓ Fast lookup using index
- ✓ ~60–80% faster on large datasets
- ✓ Avoids unnecessary materialization in engines like BigQuery, Synapse, and Databricks

Best Practices:

Prefer EXISTS when:

- Dealing with large subqueries
- You want short-circuit evaluation
- NULLs may be present in subquery
- Your subquery is correlated to outer query

Use IN when:

- List is small and static
- No NULL values in subquery
- Execution plan shows good performance

Avoid both if:

- You can rewrite with JOINS or CTEs
 - You're running the query in a distributed engine (use LEFT SEMI JOIN in Spark)
-